Full-Stack Interview Cheatsheet (Node.js • React • Express)
— Quick defs, ELI5, code & twisters —

Generated on 2025-09-02 04:32 (IST)
For: Niskarsh — condensed from recent prep chats

This cheat sheet condenses definitions, ELI5 explanations, working snippets & twisters from our prep.

Node.js — Event Loop & Task Ordering

DEFINITION (short): In Node.js, user code runs → microtasks → event-loop phases. Within microtasks,
process.nextTick() runs before Promise callbacks. Then timers (setTimeout/Interval), then other phases,
and setImmediate() runs in the 'check' phase after timers.

ELI5: Think of a two-step “fast lane” that runs before the regular road:
(1) nextTick queue (VIP), (2) Promise microtask queue (Premium). After both are empty, traffic moves to
the big roundabout (timers → poll → check → close).

Golden order: nextTick → Promise.then → setTimeout(0) → setImmediate (usually).

Code demo + expected output:
```
  console.log('A');
  setTimeout(() => console.log('t0'), 0);
  setImmediate(() => console.log('immediate'));
  process.nextTick(() => console.log('nextTick'));
  Promise.resolve().then(() => console.log('promise'));
  // Output:
  // A
  // nextTick
  // promise
  // t0
  // immediate
```

TWISTERS:
- Why can nextTick starve timers? If you keep queuing nextTicks inside nextTick, the VIP lane never ends.
- setTimeout(0) vs setImmediate? After I/O, setImmediate often fires first; otherwise timers usually do.

Node.js — Streams & Backpressure

DEFINITION: Streaming processes data piece-by-piece (Readable/Writable/Duplex/Transform). Backpressure
happens when the consumer is slower than the producer, so buffers fill up.

ELI5: A faucet (Readable) fills a cup (Writable). If you pour too fast, it overflows. Backpressure is the
signal to slow the faucet until the cup catches up.

Idiom 1 — pipe() handles backpressure automatically:
```
  const rs = fs.createReadStream('bigfile');
  const ws = fs.createWriteStream('copy');
```

```
  rs.pipe(ws); // pipe pauses/resumes under the hood

Idiom 2 — manual control with 'drain':
  rs.on('data', chunk => {
    if (!ws.write(chunk)) {
      rs.pause();
      ws.once('drain', () => rs.resume());
    }
  });

Pro tips:
- Use stream.pipeline(...) (or promises version) to wire streams with proper error +
backpressure handling.
- For HTTP uploads/downloads, always prefer streaming over buffering to avoid memory blowups.


## React — useEffect vs useLayoutEffect

DEFINITION: useEffect runs after the browser paints (async), good for non-blocking work (fetch,
log,
subscriptions). useLayoutEffect runs synchronously after DOM mutations but before paint; use it
when you
must read layout (getBoundingClientRect) or adjust DOM synchronously to avoid flicker.

ELI5: useEffect = “after the screen updates.” useLayoutEffect = “freeze, measure/adjust, then
show.”

Gotcha — infinite render loop:
  const [count, setCount] = useState(0);
  useEffect(() => setCount(count + 1), [count]); // □ runs forever
Fixes:
- Run once: useEffect(() => setCount(c => c + 1), [])
- Guard:    useEffect(() => { if (count < 5) setCount(c => c + 1); }, [count])


## React — Portals

DEFINITION: A Portal renders children into a DOM node outside the parent’s DOM hierarchy while
staying
in the same React tree (context & event bubbling still work).

Typical uses: Modals, tooltips, dropdowns, toasts — anything that should visually escape parent
stacking.

Example:
  import { createPortal } from 'react-dom';
  function Modal({ children }) {
    return createPortal(children, document.getElementById('modal-root'));
  }

Note: Events bubble through React parents even if DOM parents differ.


## Express — vhost & Helmet
```

vhost (virtual host):
- Route requests to different Express apps based on the Host header (e.g., admin.example.com vs blog.example.com).
```
  const app = express();
  app.use(vhost('admin.example.com', adminApp));
  app.use(vhost('blog.example.com', blogApp));
```

Helmet (security headers):
- Sets secure HTTP headers (CSP, X-Frame-Options, X-Content-Type-Options, HSTS, Referrer-Policy, etc.).
- One-liner hardening: app.use(helmet());
- Tweak policies for CDNs, inline scripts, etc.

Production tips:
- Put Express behind a reverse proxy (Nginx/ALB), enable trust proxy as needed.
- Use rate limiting & robust input validation; combine Helmet with CSRF protection when relevant.

Quick Reference — Bite-size Reminders

EVENT LOOP CHEAT:
- Priority: nextTick > Promise microtasks > timers > check(setImmediate)
- Avoid nextTick waterfalls (starvation). Prefer microtasks for “post-render” scheduling in frontends.

STREAMS CHEAT:
- Prefer pipeline(...). Let pipe/pipeline manage pause/resume & errors.
- Watch write() return value; listen for 'drain'.

REACT CHEAT:
- useEffect: after paint; useLayoutEffect: before paint.
- Guard state updates inside effects to avoid loops.
- Portals: render outside DOM hierarchy; still same React tree.

EXPRESS CHEAT:
- vhost for multi-domain routing in one server.
- Helmet for security headers; configure CSP carefully.

Interview Twisters — With Answers

1) Output?
```
  console.log('A');
  setTimeout(() => console.log('t0'), 0);
  setImmediate(() => console.log('immediate'));
  process.nextTick(() => console.log('nextTick'));
  Promise.resolve().then(() => console.log('promise'));
```
Answer: A → nextTick → promise → t0 → immediate

2) Why can nextTick starve timers?
Answer: nextTick runs before advancing phases; recursively queuing nextTicks prevents the loop from moving on.

3) How do you stop a useEffect loop?

Answer: Use an empty dependency array (run once) or a condition before setState, or compare refs/values.

4) Backpressure signal on write streams?
Answer: write() returns false; wait for 'drain' then resume pushing data.